

FAST

FORMAÇÃO ACCELERADA EM
SOLUÇÕES DE TECHDESIGN



C.E.S.A.R

Testes de API

Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.



RestAssured e Automação dos testes

Testes de API
Aula 2



O que veremos hoje?

- Revisão da aula anterior
- O que é RestAssured e como configurar o ambiente
- Composição de um teste automatizado com RestAssured
- Requisições HTTP e como validá-las
- Validações robustas em API
- Autenticação em Testes Automatizados

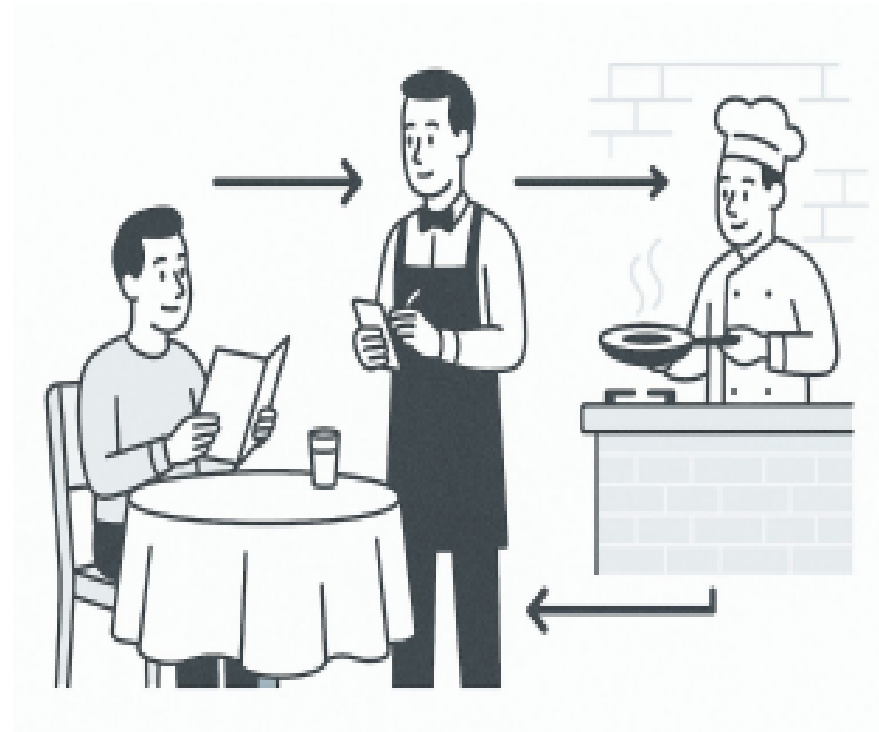
Revisão da aula anterior

Revisão

→ API é um **conjunto de regras e protocolos** que permite que diferentes softwares se comuniquem sem precisar saber como eles foram implementados.

É um contrato que define claramente:

- O que podemos pedir
- Como devemos pedir
- O que vamos receber



Revisão

Importância do contrato estabelecido pelas APIs:

- **Previsibilidade:** Garante que o sistema que usa a API saiba exatamente o que esperar do outro.
- **Estabilidade:** Se o "contrato" for alterado, ambas as partes precisam estar cientes para não "quebrar" a comunicação.
- **Qualidade:** Nos testes de API, nós verificamos se a API está cumprindo seu contrato.

Importância de validar a consistência deste contrato (testar as APIs):

- A API é o "Coração" do Software
- Detecção Precoce de Bugs
- Qualidade Robusta
- Agilidade

Revisão

HTTP (HyperText Transfer Protocol) é um protocolo usado para comunicação entre clientes e servidores. É uma **requisição HTTP** é enviada por um cliente para pedir ou enviar dados ao servidor. Ela é composta por:

- **Método:** define o tipo da ação (GET, POST, PUT, DELETE, etc.)
- **URL:** o endereço do recurso
- **Cabeçalhos (headers):** informações adicionais como autenticação, tipo de conteúdo, etc.
- **Corpo (body):** usado em métodos como POST/PUT para enviar dados (por exemplo, um JSON)

Revisão

Scripts no Postman

- Usamos o objeto global **pm** (que o Postman oferece) e o **pm.test()** para criar nossas validações.
- A sintaxe é simples e focada em "o que esperamos":
 - `pm.test("Meu teste de status", function () { ... });`
 - `pm.response.to.have.status(200);` (Valida o Status Code)
 - `pm.expect(pm.response.json().message).to.eql("Sucesso!");` (Valida um texto na resposta JSON)

O que é RestAssured e como configurar o ambiente



O que é RestAssured?

RestAssured é uma biblioteca Java para testes de APIs REST. Ela facilita a criação de testes automatizados HTTP com suporte a:

- Métodos `GET`, `POST`, `PUT`, `DELETE`, etc.
- Validação de status, cabeçalhos e body
- Integração com `JUnit` ou `TestNG`
- Autenticação (Basic, OAuth, etc)



Por que usar o RestAssured?

→ Sintaxe **fluida** e **legível** (estilo Gherkin)

- `given().`
- `queryParam("email", "API_Tester2025@qa.com.br").`
- `when().`
- `get("/usuarios").`
- `then().`
- `statusCode(200).`
- `body("usuarios[0].nome", equalTo("API Tester da Silva"));`

→ Ideal para **QA/DEVs** que usam Java e precisam testar APIs

→ Boa integração com frameworks de testes e **CI/CD**

Como configurar o ambiente para usar RestAssured?

Requisitos:

- Java JDK 8 ou superior
- Maven (ou Gradle)
- IDE Java (IntelliJ, Eclipse, VsCode)

Como configurar o ambiente para usar RestAssured?

Configuração com o Maven:

Dependência do **RestAssured** no Pom.xml

- o `<!-- RestAssured -->`
- o `<dependency>`
- o `<groupId>io.rest-assured</groupId>`
- o `<artifactId>rest-assured</artifactId>`
- o `<version>5.4.0</>`
- o `<scope>test</>`
- o `</dependency>`

Como configurar o ambiente para usar RestAssured?

Configuração com o Maven:

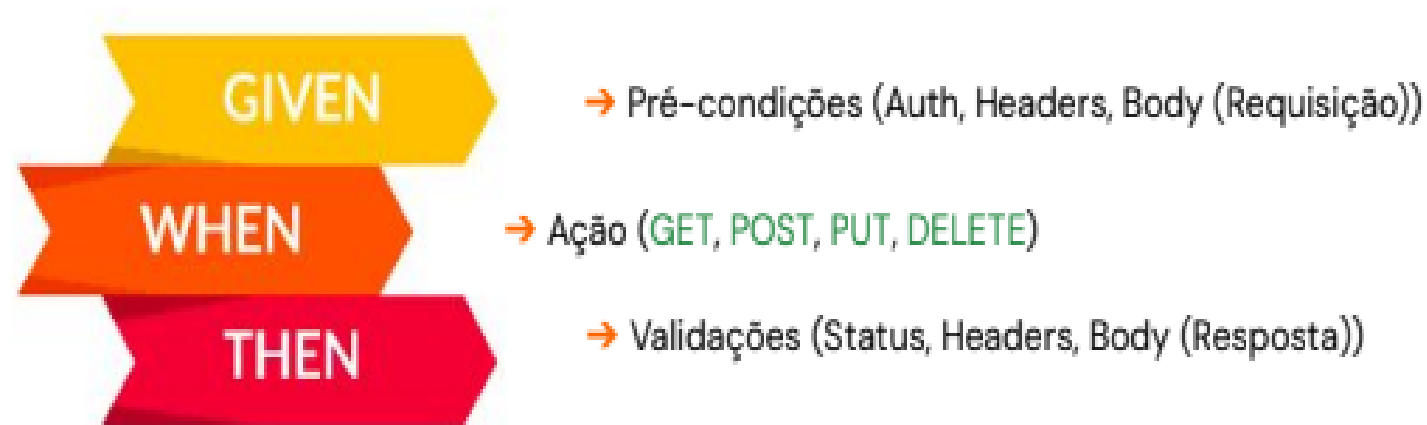
Dependência do **JUnit** no Pom.xml

- `<!-- JUnit (para testes) -->`
- `<dependency>`
- `<groupId>org.junit.jupiter</groupId>`
- `<artifactId>junit-jupiter</artifactId>`
- `<version>5.10.0</version>`
- `<scope>test</scope>`
- `</dependency>`

Composição de um teste automatizado com RestAssured

Estrutura do teste

Estrutura Básica do Código RestAssured



Estrutura de Projeto Recomendada

- /src
 - └─ /main
 - └─ /java
 - └─ /tests
 - └─ /api
 - └─ UserApiTest.java
- /pom.xml

Prática – O que iremos usar?

Aqui veremos como é a estrutura mais simples de testes para requisições tipo **GET** e **POST**. Vamos configurar e executar o primeiro teste de API com o RestAssured juntos?



Do que vamos precisar?

→ **Replit** (<https://replit.com>)

→ **API:** (<https://serverest.dev/>)

Configuração inicial

- Acesse: <https://replit.com>
- Clique em: **Sign up** (canto superior direito)
- Selecione: **Continue with Google** (ou outra opção de login)
- Escolha sua conta Google e conceda as permissões.
- Pronto! Sua conta está criada.

Clonar um novo projeto com Maven

- Após o login, acesse o [Link do projeto](#) enviado no chat
- Clique no botão **Remix this app**
- Dê um nome ao seu projeto e clique **Remix App**

Estrutura do teste

Exemplo Simples (GET)

```
given().  
when().  
    get("/users/1").  
then().  
    statusCode(200).  
    body("username", equalTo("Bret"));
```

Estrutura Completa com BaseURI

```
given().  
    header("Content-Type", "application/json").  
    body("{ \"title\": \"foo\", \"body\": \"bar\", \"userId\": 1 }").  
when().  
    post("/posts").  
then().  
    statusCode(201).  
    body("title", equalTo("foo"));
```

Estrutura do teste

Exemplo da estrutura de um teste completo

```
src > main > tests > UserTest.java
1 import io.restassured.RestAssured;
2 import org.junit.jupiter.api.Test;
3 import static io.restassured.RestAssured.*;
4 import static org.hamcrest.Matchers.*;
5
6 public class UserTest {
7
8     @Test
9     public void testGetUsers() {
10
11         RestAssured.baseURI = "https://serverest.dev";
12
13         System.out.println("Iniciando o teste.");
14
15         try {
16             given().
17             when().
18                 get("/usuarios").
19             then().
20                 statusCode(200).
21                 body("usuarios.name", hasItem("Fulano da Silva"));
22
23             System.out.println("Teste passou!");
24         }
25         catch (AssertionError e) {
26             System.out.println("Teste falhou!" + e.getMessage());
27         }
28     }
29 }
```

Prática (Consulta do Contrato)

POST /usuarios Cadastro usuário

Não é permitido cadastrar usuário com email já utilizado

Parameters Cancel

Name	Description
USER required	Body Value Model
object (body)	<pre>{ "nome": "API Tester da Silva", "email": "API_Tester0025@qa.com.br", "password": "APItest", "administrador": "true" }</pre>

Response Response content type: application/json

Curl

```
curl -X 'POST' \
  "https://serverest.dev/usuarios" \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "nome": "API Tester da Silva",
    "email": "API_Tester0025@qa.com.br",
    "password": "APItest",
    "administrador": "true"
  }'
```

Request URL:

https://serverest.dev/usuarios

Server response

Code Details

201 Response body

```
{
  "message": "Cadastro realizado com sucesso",
  "_id": "100000000abv0zki"
}
```

Download

INTERVALO!



20 min.

Ainda sobre a prática

Validações comuns:

1. → Como validar o código HTTP?
`.statusCode(201)`
2. → Como validar o corpo da resposta?
`.body("campo", equalTo("valor"))`
3. → Como validar caminho no JSON da resposta?
`.body("address.city", equalTo("Gwenborough"))`
4. → Como validar Arrays e Listas?
`.body("size()", equalTo(10))`
5. → Como validar cabeçalhos?
`.header("Content-Type", equalTo("application/json"))`
6. → Como validar tempo de resposta?
`.time(lessThan(2000L))`

Requisições HTTP e como validá-las

Requisições HTTP

GET - Obter informações

GET `/usuarios` Listar usuários cadastrados

POST - Enviar dados

POST `/usuarios` Cadastrar usuário

PUT - Atualizar informações

PUT `/usuarios/{_id}` Editar usuário

DELETE - Remover dados

DELETE `/usuarios/{_id}` Excluir usuário

GET

Requisição utilizada para solicitar dados de um recurso específico de um servidor web.

Tipo	Status Codes	O que validar
GET	200, 404	Dados, estrutura, tempo, headers

👍 Boas práticas de validação:

- Validar que o tempo de resposta é rápido (de acordo com o nicho da aplicação) e estável
- Validar cenários positivos (ex.: buscar um usuário existente)
- Validar cenários negativos (ex.: buscar um ID inexistente: 404 ou mensagem de erro)
- Verificar se listas retornam elementos esperados (quantidade, filtros, paginação)

POST

Requisição utilizada para enviar dados para um servidor web, geralmente para criar recursos.

Tipo	Status Codes	O que validar
POST	201, 400	Criação, ID gerado, persistência

👍 Boas práticas de validação:

- Validar envio com todos os campos obrigatórios → sucesso
- Testar envio com campos faltando → erro 400
- Testar envio com dados inválidos (e-mails inválidos, números negativos, etc.)
- Validar duplicidade, se a API permite ou não
- Validar a persistência da operação, realizando um GET após o POST.

PUT

Requisição utilizada para atualizar ou substituir um recurso existente no servidor.

Tipo	Status Codes	O que validar
PUT	200, 204, 400, 404	Atualização correta, idempotência

👍 Boas práticas de validação:

- Testar atualização total (PUT) ou parcial (PATCH)
- Validar comportamento ao tentar atualizar um recurso inexistente → deve retornar erro 404
- Testar atualização com payload inválido (dados da requisição errados, tipos incorretos)
- Validar que caso a mesma chamada seja executada várias vezes o resultado será sempre o mesmo

DELETE

Requisição utilizada para solicitar que um recurso específico seja removido do servidor.

Tipo	Status Codes	O que validar
DELETE	204, 200, 404	Remoção, idempotência, persistência após DELETE

👍 Boas práticas de validação:

- Validar deleção de recursos existentes;
- Validar tentativa de deletar recursos já excluídos ou inexistentes;
- Confirmar que deleções não afetam recursos não relacionados (integridade dos dados);
- Validar que deletar duas vezes o mesmo registro não deve causar um erro grave.

Validações robustas em API

Validações em requisições do tipo GET

Teste 1 — GET Produtos: Listar os produtos disponíveis.

```
1  @Test
2  public void testGetAllProducts() {
3      RestAssured.baseURI = "https://serverest.dev";
4
5      given().
6      when().
7          get("/produtos").
8      then().
9          statusCode(200).
10         header("Content-Type", containsString("application/json")).
11         body("quantidade", greaterThan(0)).
12         body("produtos[0].nome", notNullValue()).
13         time(lessThan(2000L)); // Tempo de resposta menor que 2s
14 }
```

Métodos auxiliares

Auxiliar 1 — Método para criação de usuário Admin (necessário para poder autenticar e gerenciar produtos)

```
30 public void createUser() {
31     RestAssured.baseURI = "https://serverest.dev";
32
33     String requestBody = ""
34     {
35         "nome": "QA Teacher",
36         "email": "qa.teacher8@test.com",
37         "password": "1234",
38         "administrador": "true"
39     }
40     "";
41
42     given().
43         header("Content-Type", "application/json").
44         body(requestBody).
45     when().
46         post("/usuarios").
47     then().
48         statusCode(201);
49 }
```

Métodos auxiliares

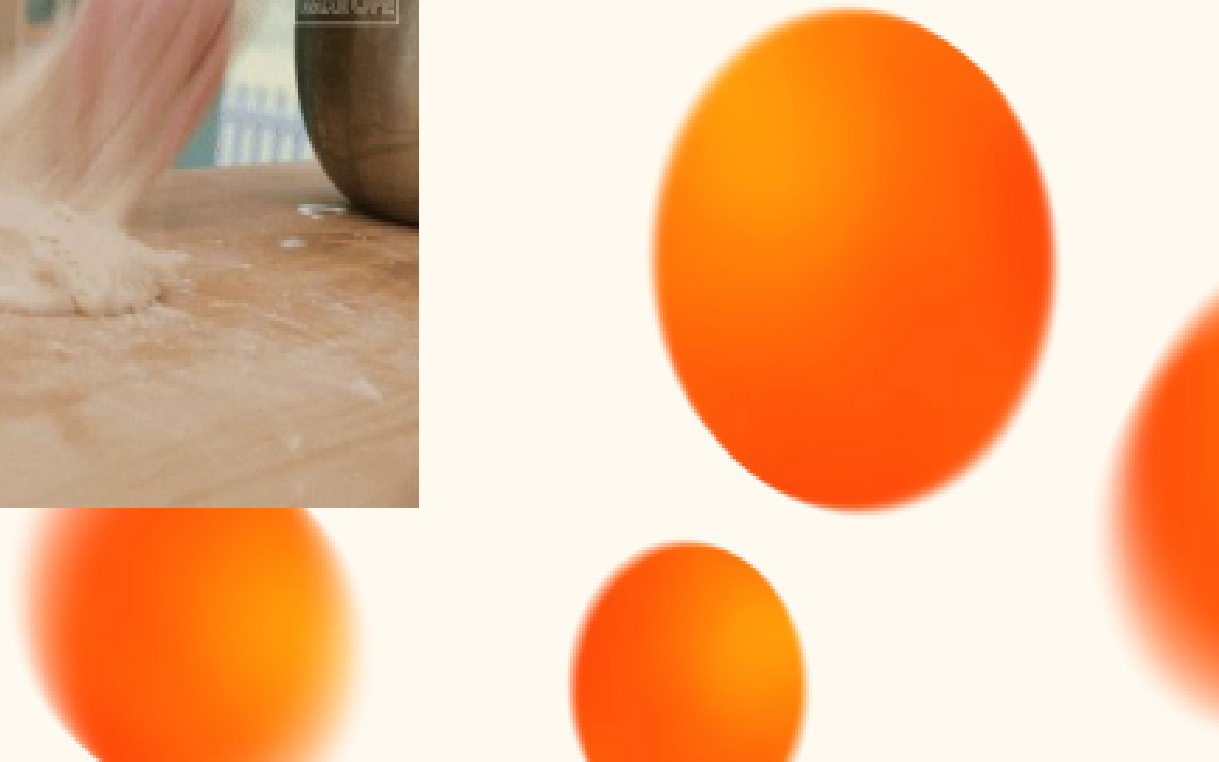
Auxiliar 2 — Método para realizar Login e obter o Token de autenticação

```
51 public String loginAndGetToken() {
52     RestAssured.baseURI = "https://serverest.dev";
53
54     String requestBody = """
55     {
56         "email": "qa.teacher8@test.com",
57         "password": "1234"
58     }
59     """;
60
61     String token =
62         given()
63             .header("Content-Type", "application/json").
64             .body(requestBody).
65         when()
66             .post("/login").
67         then()
68             .statusCode(200).
69             .extract()
70             .path("authorization");
71
72     System.out.println("Token recuperado" + token);
73
74     return token;
75 }
```

Mãos na massa (de novo)!

Aqui veremos como podem ser implementadas validações para requisições do tipo POST e PUT:

- Teste 1 — POST Produtos: Criar um novo produto.
- Teste 2 — PUT Produtos: Atualizar um produto existente.



Validações em requisições POST

Teste 2 — POST Produtos: Criar um novo produto.

```
8  @Test
9  public void testCreateProduct(String token) {
10     RestAssured.baseURI = "https://serverest.dev";
11
12     String requestBody = """
13     {
14         "nome": "Logitech MX Vertical Aula API FAST",
15         "preco": 470,
16         "descricao": "Mouse Rápido",
17         "quantidade": 100
18     }
19     """,
20
21     try {
22         given()
23             .header("Content-Type", "application/json")
24             .header("Authorization", token)
25             .body(requestBody)
26         .when()
27             .post("/produtos")
28         .then()
29             .statusCode(201)
30             .body("message", equalTo("Cadastro realizado com sucesso"))
31             .body("_id", notNullValue())
32             .time(lessThan(3000L));
33
34         System.out.println("Teste passou!");
35
36     } catch (AssertionError e) {
37         System.out.println("Teste falhou!" + e.getMessage());
38     }
39 }
```

Validações em requisições PUT

Teste 3 — PUT Produtos: Atualizar um produto existente.

```
46 @Test
47 public void testUpdateProduct(String token, String productId) {
48     RestAssured.baseURI = "https://serverest.dev";
49
50     System.out.println("🔴 Iniciando teste: PUT /produtos/" + productId);
51
52     String updatedBody = """
53         {
54             "nome": "Monitor Gamer API Test UPDATED 6",
55             "preco": 1099,
56             "descricao": "Produto atualizado em aula do FAST",
57             "quantidade": 10
58         }
59         """;
60
61     try {
62         given().
63             header("Content-Type", "application/json").
64             header("Authorization", token). // <- o token criado
65             body(updatedBody).
66         when().
67             put("/produtos/" + productId).
68         then().
69             log().all().
70             statusCode(200).
71             body("message", equalTo("Registro alterado com sucesso")).
72             time(lessThan(2000L));
73
74         System.out.println("✅ Teste passou!");
75
76     } catch (AssertionError e) {
77         System.out.println("❌ Teste falhou!");
78         System.out.println("Motivo: " + e.getMessage());
79     }
80
81     System.out.println("🏁 Fim do teste.");
82 }
```

Validações em requisições DELETE

Teste 4 — DELETE Produtos: Deletar um produto existente.

```
84     @Test
85     public void testDeleteProduct(String token, String productId) {
86         RestAssured.baseURI = "https://serverest.dev";
87
88         System.out.println("🕒 Iniciando teste: DELETE /produtos/" + productId);
89
90         try {
91             given().
92                 header("Authorization", token). // <- o token criado
93             when().
94                 delete("/produtos/" + productId).
95             then().
96                 log().all().
97                 statusCode(200).
98                 body("message", equalTo("Registro excluído com sucesso")).
99                 time(lessThan(2000L));
100
101             System.out.println("✅ Teste passou!");
102
103         } catch (AssertionError e) {
104             System.out.println("❌ Teste falhou!");
105             System.out.println("Motivo: " + e.getMessage());
106         }
107
108         System.out.println("🏁 Fim do teste.");
109     }
```

Autenticação em testes automatizados

Autenticação em API

Autenticação em APIs é o processo de garantir que quem faz uma requisição é uma entidade autorizada

→ APIs frequentemente requerem autenticação para proteger dados sensíveis e restringir acessos não autorizados.

Principais Tipos de Autenticação em APIs

1 Sem Autenticação (Pública)

- API aberta, qualquer um pode consumir.
 - Ex.: APIs de dados públicos como OpenWeatherMap, IBGE.

Autenticação em API

2 Basic Authentication (Autenticação Básica)

- Usa uma combinação de **usuário + senha**, codificados em **Base64**.
- Enviados no cabeçalho HTTP:

`Authorization: Basic <Base64(usuario:senha)>`

Exemplo RestAssured - Basic Auth

- `given().`
- `auth().preemptive().basic("usuario, "senha").`
- `when().`
- `get("/minha-api").`
- `then().`
- `statusCode(200);`

 **Problema:** pouco segura sem HTTPS.

Autenticação em API

3 API Key (Chave de API)

- A chave é enviada na URL ou no header.

`GET /produtos?apikey=123456789`

ou

`Authorization: ApiKey 123456789`

Exemplo RestAssured - API Key

- `given().`
- `queryParam("apikey", "123456789").`
- `when().`
- `get("/minha-api").`
- `then().`
- `statusCode(200);`

✓ Simples de implementar.

✗ Menos segura se não for usada sobre HTTPS.

Autenticação em API

4 Token (JWT, Bearer Token)

- Cliente faz login (POST com usuário/senha).
- API responde com um **token** (geralmente JWT).
- As próximas requisições incluem esse token no header:

- Authorization: Bearer <colar_o_token_aqui>

Exemplo RestAssured - Basic Auth

- given().
- header("Authorization", "Bearer colar_o_token_aqui").
- when().
- get("/minha-api").
- then().
- statusCode(200);

✓ Mais seguro, mais comum em APIs modernas.

Autenticação em API

Quadro comparativo dos tipos de Autenticação:

Tipo	Segurança	Facilidade	Uso Comum
Pública	✗	✓	Dados públicos
Basic Auth	⚠	✓	APIs internas, legadas
API Key	⚠	✓	APIs simples, SaaS
Token (JWT)	✓	⚠	APIs modernas, microserviços

Autenticação em API

Exemplo de Método Utilitário para Token:

```
○ public String getAuthToken() {  
○     return  
○         given().  
○             contentType("application/json").  
○             body("{ \"email\": \"user@test.com\", \"password\": \"1234\" }").  
○             when().  
○                 post("/login").  
○             then().  
○                 statusCode(200).  
○                 extract().  
○                 path("authorization");  
○ }
```

Autenticação em API

Uso do Método Utilitário no teste:

- `String token = getAuthToken();`
-
- `given().`
- `header("Authorization", token).`
- `when().`
- `get("/produtos").`
- `then().`
- `statusCode(200);`

Autenticação em API

Boas práticas com autenticação em testes:

-  Nunca expor senhas ou tokens fixos no código.
-  Gerar tokens de forma dinâmica antes dos testes.
-  Validar que endpoints sem autenticação retornam corretamente **401 Unauthorized** ou **403 Forbidden**.

**DÚVIDAS?!
:)**





C . E . S . A . R

MUITO OBRIGADO!

© CESAR | Todos os direitos reservados